Alright, let's break this down like a solid classroom session 👩‍🏫 —clear, practical, and interview-ready.

🔹 **Nullable Types & Null Safety in C#**

Null handling is one of the **most important concepts in modern C#**. If you understand this well, you'll avoid a huge class of runtime bugs (especially NullReferenceException).

🧠 **1. What is null in C#?**

null means:

"This variable does not point to any object or value."

```
string name = null; // No value assigned
```

⚠️ Problem:

```
Console.WriteLine(name.Length); // 💥 NullReferenceException
```

🔹 **2. Nullable Types (Value Types)**

❓ **Problem**

Value types (like int, double, bool) **cannot be null by default**

```
int age = null; // ❌ Compile-time error
```

✅ **Solution: Nullable Value Types**

```
int? age = null;
```

👉 int? is shorthand for:

```
Nullable<int> age = null;
```

🔍 **Example**

```
int? age = null;
```

```
if (age.HasValue)
{
    Console.WriteLine(age.Value);
}
else
{
    Console.WriteLine("No age provided");
}
```

🔧 **Key Properties**

Property	Description
HasValue	Checks if value exists
Value	Gets actual value
GetValueOrDefault()	Returns value or default

✅ **Cleaner Way**

```
int? age = null;
Console.WriteLine(age ?? 0); // Output: 0
```

🔹 **3. Nullable Reference Types (C# 8+)**

Before C# 8:

- All reference types could be null → unsafe

After C# 8:

- You can **enable null safety checks**

✅ Enable Nullable Context

#nullable enable

or in project file:

<Nullable>enable</Nullable>

🔍 Example

string name = null; // ⚠️ Warning

string? name = null; // ✅ Allowed

📌 Key Idea

Type	Meaning
string	NOT NULL
string?	CAN be null

🔹 4. Null Safety Features

Now let's go into powerful operators 🔥

1️⃣ Null-Coalescing Operator (??)

```
string? name = null;


string result = name ?? "Default Name";
Console.WriteLine(result);
```

👉 Output: Default Name

2️⃣ Null-Coalescing Assignment (??=)

```
string? name = null;

name ??= "Guest";
Console.WriteLine(name);
```

3️⃣ Null-Conditional Operator (?.)

Prevents crashes when accessing members:

```
string? name = null;

Console.WriteLine(name?.Length); // No exception
```

4️⃣ Null-Forgiving Operator (!)

Tells compiler:

“Trust me, this is NOT null”

```
string? name = GetName();
Console.WriteLine(name!.Length);
```

⚠️ Use carefully — you can still get runtime exceptions!

5️⃣ Safe Navigation Example

```
class Customer
{
    public string? Name { get; set; }
}

Customer? customer = null;
```

```
Console.WriteLine(customer?.Name);
```

🔹 5. Real-Time Example (Interview Level)

🎯 **Scenario: User Profile**

```
class User
{
    public string Name { get; set; } = string.Empty;
    public int? Age { get; set; }
}

User user = new User();

Console.WriteLine(user.Age ?? 18); // Default age
```

🔹 7. Best Practices

✅ Always enable nullable reference types
✅ Prefer string? when null is expected
✅ Use ?? for defaults
✅ Use ?. for safe access
✅ Avoid unnecessary !
✅ Initialize properties

🔥 Final Summary

- **Nullable Value Types** → int?, bool?
- **Nullable Reference Types** → string?

- **Operators**
  - ?? → default value
  - ?. → safe access
  - ??= → assign if null
  - ! → ignore warning